

Laudo de Instrumentação Multi-Provider e Estimativa de Custo da Avaliação Final

Projeto Assistente de Editais — Branch `develop` — Abril 2026

1. Contexto

O laudo 08 consolidou a arquitetura final do agente — chunker com preservação total, escopo de sessão fixado pela interface, conjunto reduzido de cinco ferramentas e self-check determinístico — e identificou como próximo passo natural a aplicação de uma bateria de avaliação multi-edital pós-correções, comparando modelos de diferentes portes para mensurar a sensibilidade do sistema à escolha do gerador.

Este laudo documenta o conjunto de ajustes operacionais realizados para viabilizar essa avaliação. Quatro frentes foram tratadas: a habilitação de quatro provedores de LLM no aplicativo Streamlit, com auto-troca do agente em sessão; um *toggle* de memória do chat para isolamento de variáveis na bateria; um pequeno pós-processamento de referências para corrigir um sintoma menor observado nas respostas; e uma instrumentação dedicada de avaliação em notebook, com captura de tokens provider-agnóstica e extrapolação de custo. Ao final, fixa-se o orçamento estimado para a bateria completa e justifica-se o conjunto de modelos retidos para a comparação.

2. Toggle de memória do chat

A discussão de avaliação em batch motivou uma pergunta operacional: o agente deve ser avaliado com ou sem histórico cumulativo entre as 50 perguntas? As duas modalidades testam coisas diferentes. Sem histórico, cada pergunta exercita o pipeline completo (system prompt + retrieval + geração) de forma independente — é o regime que mede capacidade fria, isolada de contexto. Com histórico, mede-se o comportamento real do produto na boca do usuário, incluindo a tendência (já documentada no laudo 08) do LLM reaproveitar parâmetros antigos por inércia.

Para permitir os dois regimes sem fragmentar a interface, foi acrescentado um *toggle* na barra lateral do Streamlit, "Memória do chat", ligado por padrão. Quando desligado, cada pergunta é encaminhada ao agente com `chat_history=[]`, ignorando completamente as mensagens anteriores. As mensagens permanecem visíveis na tela porque o render do histórico é independente do que é passado ao agente — são dois canais separados que compartilham apenas a lista `st.session_state.messages`.

A implementação é uma única alteração no `app.py`: a chamada a `ask()` lê o toggle e seleciona entre `st.session_state.messages[:-1]` (memória ligada) e `[]` (desligada). Nada

precisou ser tocado no `agent.py`, no `Agent.ask()` ou nas tools — a separação entre o estado da UI e o estado do agente já estava bem estabelecida desde a fixação do escopo na sidebar.

A modalidade adotada para a avaliação batch deste laudo foi memória off, justamente para isolar capacidade do gerador da capacidade de manter contexto.

3. Habilitação multi-provider

A rotina de avaliação exige comparar modelos de portes diferentes em vários provedores. O `llm_factory.get_llm` original suportava apenas OpenAI e Google. O `app.py` listava esses dois provedores no seletor da barra lateral e oferecia uma seleção de modelos compatível.

A extensão foi feita em três pontos coerentes.

3.1 Configurações

O `Settings` em `config.py` recebeu dois campos novos — `anthropic_api_key` e `groq_api_key` — sem alterar os existentes. Como o `pydantic-settings` já estava configurado com `extra="ignore"`, a presença das chaves no `.env` não dependia de novos campos para ser aceita; a adição explícita é uma documentação operacional do que o sistema espera consumir.

3.2 Factory

Dois novos branches em `get_llm`, um para `anthropic` (importando `langchain_anthropic.ChatAnthropic`) e outro para `groq` (importando `langchain_groq.ChatGroq`). Cada branch segue o mesmo padrão dos existentes: instanciar o cliente com modelo, temperatura e a chave de API correspondente. A função `list_available_models` foi atualizada para refletir os modelos sugeridos por provedor.

3.3 Interface e auto-troca de agente

O `provider_options` no `app.py` foi expandido para a lista completa de quatro provedores, e o `model_options` recebeu listas curadas para cada um — quatro modelos OpenAI (com `gpt-4o-mini` mantido como padrão), três Google, três Anthropic, três Groq. O cálculo do `index` default deixou de ser uma comparação binária com OpenAI e passou a ser indexado pela posição na lista, com fallback para zero caso o provedor do `.env` não esteja entre os listados.

Uma fricção operacional surgiu durante o uso: ao trocar provedor ou modelo no seletor da sidebar, o agente em `st.session_state["agent"]` continuava sendo o anterior, porque a função `get_agent()` só constrói novo agente quando `"agent" not in st.session_state`. O comportamento existia antes da expansão multi-provider, mas ficava menos visível com apenas dois provedores. A correção foi acrescentar uma comparação `(provider, model)` com o valor anterior armazenado em `st.session_state["modelo_anterior"]`; quando difere, o agente é descartado e reconstruído na próxima chamada. O histórico de mensagens é deliberadamente

preservado nessa troca, porque é útil para A/B testar o mesmo turno em modelos diferentes; quem quiser começar limpo continua usando o botão "Reiniciar conversa", que descarta agente e mensagens em conjunto.

4. Modelos retidos e modelos descartados

A escolha dos modelos a comparar partiu de três portes — pequeno, médio e grande — em cada provedor disponível, com a hipótese implícita de que os Llamas via Groq, sendo gratuitos, ofereceriam um piso de custo zero útil para enquadrar o resto.

Essa hipótese não sobreviveu ao primeiro teste prático.

Modelos considerados na bateria de avaliação	
Retidos (7 modelos)	Descartados
OpenAI · pay-as-you-go • gpt-4o-mini (pequeno · baseline) • gpt-5.4-mini (médio leve) • gpt-5.4 (grande) • gpt-5.5 (frontier) Anthropic · pay-as-you-go • claude-haiku-4-5 (pequeno) • claude-sonnet-4-6 (médio) • claude-opus-4-7 (frontier) Critério: pay-as-you-go direto, sem free tier interferindo, três portes em cada provedor. Custo estimado: ~ US\$ 38 3 editais × 50 perguntas × 7 modelos	Groq · Llama (3 modelos) llama-3.1-8b-instant llama-4-scout-17b llama-3.3-70b-versatile → free tier: TPM = 6000 por modelo → request do agente: 5–8k tokens → erro 413 reproduzível → Developer Tier indisponível ("high demand") Google · Gemini (3 modelos) gemini-2.5-flash-lite, 3-flash, 3.1-pro → free tier: dados podem ir para treinamento → versão paga viável, mas adiciona eixo de comparação sem ganho metodológico claro → deferido para próximo ciclo

Figura 1. Modelos retidos e descartados na bateria. A hipótese inicial era cobrir quatro provedores; restaram dois — OpenAI e Anthropic — após a inviabilidade técnica do Groq e a falta de ganho metodológico em adicionar Google na versão paga.

4.1 Groq foi descartado por inviabilidade técnica

O free tier do Groq impõe um limite de 6.000 tokens por minuto (TPM) por modelo, válido uniformemente para todos os Llamas oferecidos. O agente, com seu SYSTEM_PROMPT longo, doze chunks retornados pelo `buscar_no_edital` e a iteração de tool-call que reenvia toda a lista de mensagens, regularmente emite requisições individuais de 5.000 a 8.000 tokens. Em uma das tentativas, o servidor retornou erro 413 — Request too large, indicando Limit 6000, Requested 6484 : a requisição isolada já excedia o teto por minuto.

Por que o Groq foi descartado

A. Free tier não comporta o agente

Limite TPM por modelo: **6.000 tokens/min**
Request típico do agente: **5–8 mil tokens**
(`SYSTEM_PROMPT` + 12 chunks do RAG + re-envio do histórico em cada iteração)

Error code: 413 – Request too large
for model `llama-3.1-8b-instant`
on tokens per minute (TPM):

Limit 6000, Requested 6484
please reduce your message size

→ a request isolada já excede o teto
→ válido para todos os Llamas (8B, 70B, Scout)

B. Saída paga indisponível

Developer Tier
Pay per Token, limites 10× maiores
(60–120k TPM, sem cap diário)

⚠ **Aviso oficial no console:**
"Developer tier upgrades are temporarily unavailable due to high demand. We apologize."

→ caminho de escape do TPM bloqueado pelo próprio provedor
→ alterar o agente (encurtar prompt, reduzir `n_results`) descartado: agente está afinado

Figura 2. Por que o Groq foi descartado. (A) O free tier de 6.000 TPM é menor que o tamanho típico de uma única requisição do agente. (B) A migração para o Developer Tier, que daria limites operáveis (60 000–120 000 TPM), está bloqueada pelo próprio provedor.

O limite é por organização, não por chave, então criar chaves adicionais não resolveria. Trocar entre os Llamas (8B Instant, Llama 4 Scout, 3.3 70B) também não muda o teto, porque o TPM de 6 000 é o mesmo para todos no free tier. A única saída técnica seria reduzir o tamanho do request — o que exigiria mexer no agente, alterando o `SYSTEM_PROMPT` ou o `n_results` do retrieval. Como o agente está afinado e fora do escopo deste laudo, a saída foi outra.

A migração para o Developer Tier do Groq, que daria limites operáveis e desconto por uso, está atualmente bloqueada pelo próprio provedor, conforme aviso na página de billing: *"Developer tier upgrades are temporarily unavailable due to high demand"*. Sem caminho técnico para o upgrade, o Groq foi descartado da bateria.

4.2 Google foi descartado por incompatibilidade de modalidade de uso

O free tier do Gemini API é generoso em termos de chamadas, mas é explicitamente um free tier de desenvolvimento — os dados enviados podem ser usados para treinamento, e os limites de RPM ficam abaixo do necessário para uma bateria de 50 perguntas executada em sequência sem espera. A versão paga (pay-as-you-go) elimina esse problema, mas adicioná-la à bateria significa um eixo de comparação a mais por orçamento, sem ganho metodológico claro frente aos outros provedores. A decisão pragmática foi não incluir o Google na bateria deste ciclo.

4.3 OpenAI e Anthropic foram retidos

Ambos têm pay-as-you-go direto, sem free tier interferindo, com limites operacionais largos. A bateria comparativa foi planejada com sete modelos: quatro da OpenAI (`gpt-4o-mini`, `gpt-5.4-mini`, `gpt-5.4`, `gpt-5.5`) e três da Anthropic (`claude-haiku-4-5`, `claude-sonnet-4-6`, `claude-opus-4-7`). A escolha cobre os três portes em ambos os provedores, com o `gpt-4o-mini` mantido como baseline porque é o modelo já em produção no aplicativo.

5. Pós-processamento de referências numéricas para anexos

Durante os testes manuais com o `gpt-4o-mini` em modo memória off, observou-se um sintoma menor mas consistente: respostas que citavam corretamente o conteúdo de um anexo do edital frequentemente fechavam com a referência no formato `ref: edital [13]`, em lugar de `ref: edital [Anexo I]`. A causa é estrutural na saída do `buscar_no_edital`: cada grupo de chunks no retorno expõe dois rótulos alternativos para o mesmo material — o `cap_NN` herdado do chunker (em que o BNDES tem `cap_1` a `cap_12` = corpo, `cap_13` a `cap_16` = Anexos I a IV) e o `[REF: ANEXO I]` alinhado ao formato pedido pelo `SYSTEM_PROMPT`. O LLM oscilava entre as duas convenções ao gerar a referência final.

Duas soluções foram consideradas. A primeira, alterar o prompt para reforçar a regra "para anexos, usar Anexo I", já estava em vigor e não eliminava o problema. A segunda, ocultar o rótulo `cap_NN` da saída da tool, exigiria mexer no contrato entre `buscar_no_edital` e o agente — pode produzir efeitos colaterais em outras tools que dependem do mesmo metadado.

A solução adotada foi um pós-processamento puramente sintático. Após o agente retornar a resposta final, e antes de devolvê-la ao chamador, uma função `_corrigir_refs_anexos` percorre as ocorrências de `ref: edital [...]` e substitui itens que sejam inteiros puros (sem ponto) e que correspondam a um `cap_num` cujo título no TOC do edital ativo começa com `ANEXO` ou `APÊNDICE` pelo rótulo curto do anexo. Itens com ponto (`5.1`, `9.6.6`) ou já textuais (`Anexo I`) ficam intocados. No modo "Todos os editais", em que o `edital_id_ativo` é zero, o mapa fica vazio e a correção vira no-op — porque, sem edital fixado, não há como saber a qual edital o número se refere.

A escolha pelo pós-processamento se justifica pela natureza do problema: é puramente sintática, não envolve julgamento sobre conteúdo, e é localmente verificável. Mexer no prompt para resolver isso seria, em comparação, frágil e de efeito não auditável.

6. Instrumentação de avaliação

A bateria foi implementada em dois notebooks Jupyter na pasta `evals/` da raiz do projeto, junto a uma subpasta `evals/perguntas/` contendo um arquivo XLSX por edital com 50 perguntas e a categoria temática de cada uma, e a uma subpasta `evals/resultados/` em que os runs depositam suas saídas. As 50 perguntas foram derivadas de uma planilha-mãe contendo as três versões da mesma pergunta (uma por edital) com substituição automática do placeholder "X" pelo nome do concurso, garantindo equivalência semântica entre os três conjuntos.

A separação em dois notebooks reflete uma diferença técnica relevante na captura de tokens.

Captura de tokens: dois notebooks, dois mecanismos



Figura 3. Os dois notebooks de captura de tokens. À esquerda, o utilitário `get_openai_callback` da `langchain-community` cobre apenas a OpenAI via `contextvar` registrada por `langchain-openai`. À direita, um `BaseCallbackHandler` simples lê `usage_metadata` das mensagens (padrão moderno) e funciona para todos os provedores, anexado via `with_config` sem tocar no `agent.py`.

6.1 Notebook 1 — captura OpenAI-only

Usa o utilitário `langchain_community.callbacks.get_openai_callback`, que registra um handler em `contextvar` e acumula `prompt_tokens` e `completion_tokens` de toda chamada `ChatOpenAI._generate` realizada dentro do escopo `with`. Isso captura, para cada pergunta, todas as invocações que o agente faz: a invocação inicial, cada iteração do loop de tool-call (a invocação após o `ToolMessage` é a "gorda", porque carrega os doze chunks recuperados pelo RAG) e a eventual chamada do self-check. O custo em USD é calculado pelo próprio callback usando a tabela de preços que ele mantém internamente.

6.2 Notebook 2 — captura provider-agnóstica

Como o `get_openai_callback` não captura nada de provedores que não sejam OpenAI, foi implementado um `BaseCallbackHandler` simples chamado `TokenTracker` que sobrescreve `on_llm_end`. Em cada invocação, o tracker tenta primeiro `response.llm_output.token_usage` (formato antigo, OpenAI) e, em fallback, `response.generations[*].message.usage_metadata` (formato novo, padrão LangChain — adotado por `langchain_anthropic` e `langchain_google_genai`). O tracker é zerado antes de cada pergunta para isolar o consumo do turno, e é anexado aos dois LLMs do agente via `with_config(callbacks=[tracker])` — uma operação que retorna um `RunnableBinding` configurado, sem tocar no `agent.py`. O custo em USD é calculado manualmente a partir de um dicionário de preços por modelo.

6.3 Saída dos runs

Em ambos os notebooks, a saída é um XLSX por run, nomeado com {editais}__{provider}__{model}__{timestamp}.xlsx , contendo, por linha, a pergunta, a resposta do agente, os tokens de entrada e saída, o número de invocações ao LLM no turno, o custo individual em USD, a latência em segundos e o eventual erro. Após o loop, três células sumarizam a execução, salvam o arquivo e imprimem uma tabela de extrapolação que multiplica os tokens medidos pelos preços de cada modelo da lista, oferecendo uma estimativa do custo para cada um se rodasse a mesma bateria.

7. Estimativa de custo da avaliação completa

O run de referência adotado foi com anthropic/claude-haiku-4-5 sobre o BNDES, em memória off. A escolha desse modelo como base, em vez do gpt-4o-mini , segue uma observação metodológica: tokenizers de provedores diferentes contam o mesmo texto de forma diferente — o tokenizer da Anthropic, em particular, tende a contar mais que o da OpenAI. Tomar o haiku como base produz uma estimativa conservadora (teto razoável) para os modelos OpenAI e uma estimativa praticamente exata para os outros Claudes, que compartilham o tokenizer.

A figura 4 abaixo é o produto direto do notebook, ordenada do mais barato ao mais caro.



Figura 4. Estimativa de custo da bateria. Tokens medidos com o `claude-haiku-4-5` sobre o BNDES, multiplicados pelo preço de cada modelo. Modelos retidos em verde; descartados em cinza claro. O total da bateria — sete modelos retidos sobre os três editais — fica em US\$ 37,59. Considerando os caveats de tokenizer, raciocínio e variância entre editais, o intervalo provável é US\$ 30 – US\$ 50.

A bateria planejada — quatro modelos OpenAI e três modelos Anthropic, sobre os três editais, em memória off — soma 37,59 USD. Considerando os três efeitos sistemáticos (tokenizer,

tokens de raciocínio, variância entre editais), o intervalo provável é **US\$ 30 – US\$ 50**. A ordem de grandeza é confortável dentro do orçamento de pesquisa: rodar a bateria completa em todos os modelos é viável sem etapa de aprovação.

8. Estado atual e próximos passos

A branch `develop` ao final deste ciclo consolida os ajustes operacionais necessários para a bateria de avaliação multi-edital prevista no laudo 08:

1. suporte a quatro provedores de LLM no Streamlit, com auto-troca de agente em sessão;
2. *toggle* de memória do chat para isolamento de variáveis;
3. pós-processamento de referências numéricas para anexos, sem alteração do `SYSTEM_PROMPT`;
4. instrumentação em dois notebooks com captura de tokens dedicada por provedor e geração de XLSX de saída por run.

A bateria foi enquadrada em sete modelos — quatro OpenAI e três Anthropic — após o descarte motivado de Groq (limite de TPM no free tier inviável para o tamanho de request do agente; Developer Tier indisponível) e Google (free tier não compatível com a modalidade de uso, sem ganho metodológico claro em adicioná-lo na versão paga). O orçamento estimado para a bateria completa, sobre os três editais, é da ordem de US\$ 30 – US\$ 50, com base em medição direta do `claude-haiku-4-5` sobre o BNDES extrapolada pelos preços oficiais de abril de 2026.

Direções identificadas para o próximo laudo:

1. **Execução da bateria.** Rodar os sete modelos sobre os três editais, salvar os resultados padronizados em `evals/resultados/` e produzir uma análise consolidada cruzando taxa de acerto, latência média, consumo de tokens e custo por modelo.
2. **Critério de avaliação automatizada.** A coluna "gabarito" das planilhas de perguntas, hoje preenchida com a resposta de referência extraída manualmente, viabiliza um juiz LLM que classifique cada par (resposta do agente, resposta de referência) em correto / parcial / incorreto. Um modelo médio (`claude-sonnet-4-6` ou `gpt-5.4`) é candidato natural para esse papel, mantendo independência do modelo testado.
3. **Comparação com regime memória ligada.** A bateria deste ciclo é em memória off por isolamento de variáveis. Um segundo eixo, com memória ligada, mediria o efeito do histórico cumulativo no comportamento do agente — em particular, se a tendência ao reaproveitamento de parâmetros antigos identificada no laudo 08 sobrevive ao escopo de sessão fixado.